

Name:

Roll No.:

Duration: 1.5 hours

Question:	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Points:	2	1	2	1	1	1	1	2	1	1	1	2	2	2
Score:														

1. (2 points) The sigmoid and hyperbolic tangent functions are closely related and are often used as activation functions in neural networks.

$$\sigma(Aa) = \frac{D \tanh(a) + B}{C}$$

Find A , B , C , D .

Solution:

$A = 2$, $B = 1$, $C = 2$, $D = 1$

$$\sigma(2a) = \frac{\tanh(a) + 1}{2}$$

2. (1 point) Assuming $x \in \mathbb{R}$, for what value of x will the logistic function $\frac{1}{1+e^{-(wx+b)}}$ output the value 0.5. Express in terms of w and b .

Solution:

$$x = \frac{-b}{w}$$

3. (2 points) Consider the temperature-scaled softmax function defined for a logit vector $\mathbf{z} = (z_1, z_2, \dots, z_K)$ as

$$\sigma_i(\mathbf{z}; T) = \frac{\exp(z_i/T)}{\sum_{j=1}^K \exp(z_j/T)},$$

where $T > 0$ is the temperature parameter.

Which of the following statements are **correct**? (Multiple Select Question)

- A. As $T \rightarrow 0^+$, the softmax output converges to a one-hot vector assigning probability 1 to the index corresponding to $\max_i z_i$, assuming the maximum is unique.
- B. For any $T > 0$, if $z_i > z_j$ then $\sigma_i(\mathbf{z}; T) > \sigma_j(\mathbf{z}; T)$.
- C. Increasing the temperature T increases the difference between the largest and second-largest softmax probabilities.
- D. As $T \rightarrow \infty$, the softmax distribution approaches a uniform distribution over all classes.

Solution: The correct options are **(A), (B), and (D)**.

- **(A)** is correct because a small temperature amplifies differences between logits, causing the softmax to behave like an argmax.
- **(B)** is correct since the exponential function is strictly monotonic; temperature scaling does not change the ordering induced by the logits.
- **(C)** is incorrect because increasing T smoothens the distribution, reducing peakiness and hence decreasing the gap between the largest probabilities.
- **(D)** is correct since dividing logits by a very large T makes them nearly equal, resulting in a uniform distribution.

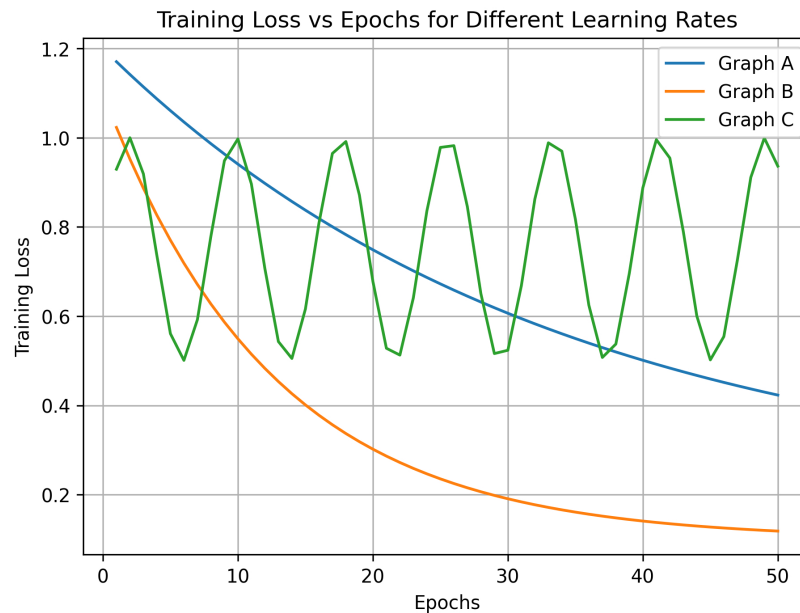
4. (1 point) Which of the following statements about **Xavier (Glorot) initialization** are correct? (*Multiple options may be correct.*)
- A. It initializes weights so that the variance of activations remains approximately constant across layers.
 - B. It is derived under the assumption of symmetric activation functions such as tanh.
 - C. For a layer with n_{in} input units and n_{out} output units, weights are sampled from $\mathcal{N}\left(0, \frac{2}{n_{\text{in}} + n_{\text{out}}}\right)$.
 - D. It helps prevent both vanishing and exploding gradients in deep networks.
 - E. Xavier initialization is optimal for ReLU activations.

Solution: The correct options are: A, B, C , D

- **Option 1 (Correct):** Xavier initialization aims to keep the variance of activations approximately constant across layers.
- **Option 2 (Correct):** It is derived assuming linear or symmetric activations such as tanh.

- **Option 3 (Correct):** In the Gaussian version, weights are sampled as $\mathcal{N}\left(0, \frac{2}{n_{in} + n_{out}}\right)$.
- **Option 4 (Correct):** By preserving variance during forward and backward passes, it mitigates vanishing and exploding gradients.
- **Option 5 (Incorrect):** Xavier initialization is not optimal for ReLU activations; He initialization is preferred.

5. (1 point) The graph below shows the training loss of a neural network across epochs for three different learning rates: **0.001**, **0.01**, and **1.0**.



1. Graph A shows smooth but very slow convergence.
2. Graph B shows fast convergence to a low training loss.
3. Graph C shows oscillations and does not converge properly.

Match each graph (A, B, C) with the most likely learning rate.

- (a) A $\rightarrow$ 0.01, B $\rightarrow$ 0.001, C $\rightarrow$ 1.0
- (b) A $\rightarrow$ 0.001, B $\rightarrow$ 0.01, C $\rightarrow$ 1.0
- (c) A $\rightarrow$ 1.0, B $\rightarrow$ 0.001, C $\rightarrow$ 0.01
- (d) A $\rightarrow$ 0.01, B $\rightarrow$ 1.0, C $\rightarrow$ 0.001

Solution: The correct answer is **option (b)**:

$$A \rightarrow 0.001, \quad B \rightarrow 0.01, \quad C \rightarrow 1.0.$$

6. (1 point) Estimate $\sqrt{4.2}$ using a 2nd-degree Taylor Polynomial centered at $a = 4$. Approximate to 2 decimal points.

Solution:

Function: $f(x) = \sqrt{x}$

Center: $a = 4$

Evaluate at: $x = 4.2$

Order: $n = 2$

Answer Key: $f(4) = 2, f'(4) = \frac{1}{4}, f''(4) = -\frac{1}{32}$

$$P_2(x) = 2 + \frac{1}{4}(x - 4) - \frac{1}{64}(x - 4)^2$$

$$P_2(4.2) = 2 + 0.25(0.2) - \frac{1}{64}(0.2)^2 \approx \mathbf{2.049375}$$

Solution: [2.047 - 2.052]

7. (1 point) Which component of the Adam optimizer is primarily responsible for “building momentum” to help the optimization process move out of small local minima?
- A. The second moment estimate (v_n , variance of gradients)
 - B. The first moment estimate (m_n , weighted moving average of gradients).
 - C. The division by $\sqrt{s_n}$.
 - D. The initialization of betas (β_1, β_2).

Solution:

The first moment (m_n) acts as a weighted moving average of gradients, providing momentum.

if β_1 is initialized to 0, then there is no momentum based update, hence β_1 is also important

Solution: B, D

8. (2 points) You are training a simple model with a loss function $L(w) = w_1^2 + 4w_2^2$. The current weight vector is $w = [1, 0.5]^T$. Using a learning rate of $\eta = 0.1$, calculate the updated weight vector w_{new} after one step of Gradient Descent.

Solution:

$$\nabla L = \left[\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2} \right] = [2w_1, 8w_2].$$

$$\text{At } w = [1, 0.5], \nabla L = [2(1), 8(0.5)] = [2, 4].$$

$$\text{Update: } w_{new} = w - \eta \nabla L = [1, 0.5] - 0.1[2, 4] = [1 - 0.2, 0.5 - 0.4] = [0.8, 0.1].$$

Solution: **[0.8, 0.1]**

9. (1 point) Consider a simple computational graph with scalar inputs $x = 2$ and $y = 3$. Intermediate nodes: $z = x \cdot y$ and output $L = z^2$. Calculate the gradient of the loss with respect to x (i.e., $\frac{\partial z}{\partial x}$).

Solution:

$$L = (xy)^2$$

$$\frac{\partial L}{\partial x} = y = 3$$

Solution: 3

10. (1 point)] Select the correct properties of norms and loss functions.
- A. The L1 norm ($\|y - y_{true}\|_1$) represents the absolute error.
 - B. The L2 norm corresponds to Euclidean distance and penalises large errors more heavily than L1.
 - C. The L0 norm is commonly used in standard gradient descent because it is easily differentiable.
 - D. L2 regularization leads to sparse models by forcing coefficients to zero.

Solution:

Solution: A,B

11. (1 point) Consider the ReLU activation function defined as:

$$f(z) = \max(0, z)$$

Compute the value of the derivative $f'(z)$ at $z = -3$.

- A. 0
- B. 1
- C. -3

D. Undefined

Solution: Since $z = -3 < 0$, we are in the region where the ReLU function outputs 0 and its derivative is also 0.

Therefore,

$$f'(-3) = 0$$

Answer: A

12. (2 points) Consider the loss function:

$$L(w) = w^2$$

You are given a learning rate $\eta = 0.5$ and the initial weight value $w = 2$. After performing one step of gradient descent, what is the new value of the weight?

Solution:

First, compute the gradient of the loss function:

$$\frac{dL}{dw} = 2w$$

Evaluate the gradient at $w = 2$:

$$\left. \frac{dL}{dw} \right|_{w=2} = 2(2) = 4$$

Using the gradient descent update rule:

$$w_{\text{new}} = w - \eta \frac{dL}{dw}$$

Substitute the given values:

$$w_{\text{new}} = 2 - 0.5(4)$$

$$w_{\text{new}} = 2 - 2 = 0$$

Answer: 0

13. (2 points) A dataset contains 1024 training samples. Consider one full pass over the dataset (one epoch) under the following two training strategies:

- Case 1: Mini-batch Gradient Descent with batch size = 128.
- Case 2: Stochastic Gradient Descent (SGD) with batch size = 1.

1. Compute the total number of parameter updates performed in one epoch for each case.
2. What is the approximate ratio of the number of updates (SGD : Mini-batch)?

Solution:

For one epoch, the number of updates equals:

$$\text{Number of updates} = \frac{\text{Total training samples}}{\text{Batch size}}$$

Case 1: Mini-batch Gradient Descent (batch size = 128)

$$\frac{1024}{128} = 8$$

So, Mini-batch Gradient Descent performs 8 updates per epoch.

Case 2: Stochastic Gradient Descent (batch size = 1)

$$\frac{1024}{1} = 1024$$

So, SGD performs 1024 updates per epoch.

Ratio (SGD : Mini-batch)

$$\frac{1024}{8} = 128 : 1$$

Final Answer:

1. (i) Mini-batch : 8 updates, (ii) SGD = 1024 updates
2. Ratio (SGD : Mini-batch) = 128 : 1

14. (2 points) Consider a 3-class classification problem with the following logits:

$$z = \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix}$$

The true class label is given in one-hot encoded form as:

$$y = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

The model uses the Softmax activation followed by the Cross-Entropy loss:

$$L = - \sum_{i=1}^3 y_i \log(\hat{y}_i),$$

where the softmax probabilities are defined as:

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^3 e^{z_j}}.$$

Compute the gradient of the loss with respect to the first logit, i.e.,

$$\frac{\partial L}{\partial z_1}.$$

Solution:

Step 1: Compute the Softmax probabilities

First compute the exponentials:

$$e^1 = 2.718, \quad e^2 = 7.389, \quad e^0 = 1$$

Compute their sum:

$$\text{Sum} = 2.718 + 7.389 + 1 = 11.107$$

Now compute the softmax probabilities:

$$\hat{y} = \left[\frac{2.718}{11.107}, \frac{7.389}{11.107}, \frac{1}{11.107} \right]$$

$$\hat{y} \approx [0.245, 0.665, 0.090]$$

Step 2: Use the known result for Softmax + Cross-Entropy

For Softmax followed by Cross-Entropy loss, the gradient simplifies to:

$$\frac{\partial L}{\partial z_i} = \hat{y}_i - y_i$$

Since the true label is:

$$y = [0, 1, 0],$$

the gradient with respect to z_1 is:

$$\frac{\partial L}{\partial z_1} = \hat{y}_1 - y_1$$

$$\frac{\partial L}{\partial z_1} = 0.245 - 0$$

$$\frac{\partial L}{\partial z_1} \approx 0.245$$

$$\boxed{\frac{\partial L}{\partial z_1} \approx 0.245}$$

15. (2 points) Consider a neural network with skip (residual) connections. The forward propagation at layer l is defined as:

$$a^{(l+1)} = a^{(l)} + F(a^{(l)}, W^{(l)})$$

where:

- $a^{(l)}$: output of layer l
- $a^{(l+1)}$: output of the next layer
- $F(\cdot)$: function applied at layer l
- $W^{(l)}$: weights of layer l
- L : loss function
- $\delta^{(l+1)} = \frac{\partial L}{\partial a^{(l+1)}}$: gradient coming from the next layer

During backpropagation, we are given:

$$\frac{\partial L}{\partial a^{(l+1)}} = \delta^{(l+1)}$$

Compute the gradient of the loss with respect to the previous layer activation:

$$\frac{\partial L}{\partial a^{(l)}} = ?$$

- A. $\delta^{(l+1)}$
- B. $\delta^{(l+1)} \cdot \frac{\partial F}{\partial a^{(l)}}$
- C. $\delta^{(l+1)} \left(1 + \frac{\partial F}{\partial a^{(l)}}\right)$
- D. $\frac{\partial F}{\partial a^{(l)}}$

Solution:

Step 1: Apply the Chain Rule

Using backpropagation and the chain rule:

$$\frac{\partial L}{\partial a^{(l)}} = \frac{\partial L}{\partial a^{(l+1)}} \cdot \frac{\partial a^{(l+1)}}{\partial a^{(l)}}$$

Substitute:

$$\frac{\partial L}{\partial a^{(l)}} = \delta^{(l+1)} \cdot \frac{\partial a^{(l+1)}}{\partial a^{(l)}}$$

Step 2: Differentiate the Forward Equation

From the forward pass:

$$a^{(l+1)} = a^{(l)} + F(a^{(l)}, W^{(l)})$$

Differentiate with respect to $a^{(l)}$:

$$\begin{aligned} \frac{\partial a^{(l+1)}}{\partial a^{(l)}} &= \frac{\partial}{\partial a^{(l)}} (a^{(l)}) + \frac{\partial}{\partial a^{(l)}} (F(a^{(l)}, W^{(l)})) \\ &= 1 + \frac{\partial F}{\partial a^{(l)}} \end{aligned}$$

Step 3: Final Expression

Substitute back:

$$\frac{\partial L}{\partial a^{(l)}} = \delta^{(l+1)} \left(1 + \frac{\partial F}{\partial a^{(l)}}\right)$$

$$\boxed{\frac{\partial L}{\partial a^{(l)}} = \delta^{(l+1)} \left(1 + \frac{\partial F}{\partial a^{(l)}}\right)}$$

Correct Option: C

16. (1 point) Suppose a model has very low training error but high validation error. Assume the dataset size is fixed. Which of the following changes are most likely to reduce the validation error? (Multiple Select)

- A. Increase model capacity further.
- B. Add L_2 regularization (weight decay).
- C. Reduce the number of parameters.
- D. Add more noise to the training labels.

Solution: Correct options: B, C.

- This is overfitting (high variance).
- Regularization reduces variance.
- Reducing parameters reduces model complexity.
- Increasing capacity worsens overfitting.
- Adding noise increases variance.

17. (1 point) Consider the function

$$f(x) = \sum_{i=1}^M a_i u(w_i^T x - b_i),$$

where $u(\cdot)$ is a sigmoid activation.

According to universal approximation results, what happens as $M \rightarrow \infty$?

- A. $f(x)$ can approximate any continuous function on a compact domain.
- B. $f(x)$ can represent only linear functions.
- C. Approximation is possible only if depth increases.
- D. Approximation requires non-sigmoid activations.

Solution: Correct option: A.

By universal approximation theorem, a single hidden layer with sufficient width can approximate any continuous function under mild conditions.

18. (1 point) For steepest descent with optimal step size η^* at each iteration, it holds that

$$\nabla L(w_{n+1})^T \nabla L(w_n) = 0.$$

What does this imply?

- A. Consecutive gradients are orthogonal.
- B. The loss is globally minimized in one step.
- C. The method guarantees quadratic convergence.
- D. The learning rate is constant.

Solution: Correct option: A.

Optimal line search makes successive gradients orthogonal. It does not guarantee one-step or quadratic convergence.

19. (3 points) Suppose the true data distribution is

$$P = \mathcal{N}(2, 1),$$

and a model assumes the distribution

$$Q = \mathcal{N}(0, 1).$$

Using cross-entropy loss

$$H(P; Q) = \mathbb{E}_P [-\log q(X)],$$

compute the cross-entropy loss.

(Answer up to 3 decimal places.)

Solution:

For a Gaussian model,

$$q(x) = \frac{1}{\sqrt{2\pi\sigma_Q^2}} \exp\left(-\frac{(x - \mu_Q)^2}{2\sigma_Q^2}\right)$$

The cross entropy is

$$H(P; Q) = \frac{1}{2} \log(2\pi\sigma_Q^2) + \frac{1}{2\sigma_Q^2} \mathbb{E}_P[(X - \mu_Q)^2]$$

Given:

$$\mu_P = 2, \quad \sigma_P^2 = 1$$

$$\mu_Q = 0, \quad \sigma_Q^2 = 1$$

We compute

$$\begin{aligned}\mathbb{E}_P[(X - \mu_Q)^2] &= \mathbb{E}_P[(X - 0)^2] = \text{Var}(X) + (\mu_P - 0)^2 \\ &= 1 + (2)^2 = 5\end{aligned}$$

Now,

$$H(P; Q) = \frac{1}{2} \log(2\pi) + \frac{1}{2}(5)$$

$$\log(2\pi) = 1.837$$

$$H(P; Q) = \frac{1}{2}(1.837) + 2.5$$

$$= 0.919 + 2.5 = 3.419$$

$$\boxed{3.419}$$

20. (3 points) Consider the quadratic loss function

$$L(w) = 5w^2.$$

Gradient descent update rule is

$$w_{n+1} = w_n - \eta \frac{dL}{dw}.$$

For what range of learning rate η will the iterations converge to $w = 0$?

Find the maximum value of η for which convergence is guaranteed.

Solution:

First compute gradient:

$$\frac{dL}{dw} = 10w$$

Update rule:

$$w_{n+1} = w_n - \eta(10w_n)$$

$$w_{n+1} = (1 - 10\eta)w_n$$

For convergence:

$$|1 - 10\eta| < 1$$

Solve inequality:

$$-1 < 1 - 10\eta < 1$$

Left side:

$$-1 < 1 - 10\eta$$

$$-2 < -10\eta$$

$$\eta < 0.2$$

Right side:

$$1 - 10\eta < 1$$

$$-10\eta < 0$$

$$\eta > 0$$

Thus,

$$0 < \eta < 0.2$$

Maximum allowable learning rate:

$$0.2$$

21. (1 point) Consider a deep neural network with L layers trained using gradient descent. The weight update for layer l is given by

$$W_{t+1}^{(l)} = W_t^{(l)} - \eta \delta^{(l)} (a^{(l-1)})^T.$$

During training, you observe that:

- Gradients in early layers shrink exponentially with depth
- Training loss still decreases steadily

Which of the following statements is **most accurate**?

- A. The network is underfitting
- B. The network is learning only shallow features
- C. Vanishing gradients prevent any useful learning
- D. The effective depth of the network is reduced
- E. The learning rate must be increased

Solution: Even though gradients vanish in early layers, later layers continue to learn and reduce the loss. Hence, the network behaves as if it were shallower than its actual depth.

Correct Answer: D

22. (1 point) A neural network achieves near-zero training loss but shows unstable validation performance across epochs.

Which of the following interventions can **improve generalization but worsen optimization**? (Select all that apply.)

- A. Increasing network depth while keeping dataset size fixed
- B. Adding dropout to hidden layers
- C. Using smaller batch sizes
- D. Switching from SGD to Adam

Solution: Increasing depth, adding dropout, and using smaller batch sizes introduce noise or complexity that can improve generalization but make optimization harder. Adam usually stabilizes optimization, and early stopping does not worsen optimization.

Correct Answers: A, B, C

23. (1 point) Consider the scalar network:

$$x \xrightarrow{w_1} z_1 \xrightarrow{\text{ReLU}} a_1 \xrightarrow{w_2} z_2 \xrightarrow{\tanh} y.$$

Given:

$$x = -1, \quad w_1 = 2, \quad w_2 = 3, \quad \frac{\partial L}{\partial y} = 4,$$

compute:

$$\frac{\partial L}{\partial w_1}.$$

Solution: Forward pass:

$$z_1 = w_1 x = 2(-1) = -2.$$

Since $z_1 < 0$, $\text{ReLU}(z_1) = 0$, so $a_1 = 0$.

The gradient is blocked by ReLU, hence:

$$\frac{\partial L}{\partial w_1} = 0.$$

Final Answer: 0

24. (1 point) At a point w^* , a neural network loss function $L(w)$ satisfies:

$$\nabla L(w^*) = 0.$$

You may assume that in a neighborhood of w^* , the loss admits the following second-order Taylor approximation:

$$L(w^* + \Delta w) \approx L(w^*) + \frac{1}{2} \Delta w^\top \nabla^2 L(w^*) \Delta w,$$

that is, the local change in the loss is governed by the **Hessian term**

$$\frac{1}{2} \Delta w^\top \nabla^2 L(w^*) \Delta w.$$

Suppose the Hessian $\nabla^2 L(w^*)$ has **one large negative eigenvalue** and **many small positive eigenvalues**. Assume Δw can be chosen along any eigenvector direction of the Hessian.

What is the nature of the point w^* ?

- A. Local minimum
- B. Saddle point with one descent direction
- C. Gradient descent will diverge immediately
- D. The loss is locally convex

Solution: Since $\nabla L(w^*) = 0$, the first-order term vanishes and the local behavior of the loss is completely determined by the Hessian term

$$\frac{1}{2} \Delta w^\top \nabla^2 L(w^*) \Delta w.$$

Because the Hessian has mixed-sign eigenvalues, the quadratic form can be positive along some directions and negative along at least one direction. Hence, the loss increases in some directions and decreases in another.

Therefore, w^* is a saddle point with one descent direction.

Correct Answer: B

25. (2 points) Consider a fully connected neural network with depth L and width n at each hidden layer. Assume backpropagation is used for training.

Which of the following statements are correct for parallel computations? (Select all that apply.)

- A. Backpropagation time complexity grows linearly with depth L
- B. Increasing depth always increases expressivity
- C. Increasing width can reduce vanishing gradients
- D. Universal approximation eliminates optimization difficulties

Solution: Backpropagation requires one backward pass per layer, leading to linear dependence on L .

Correct Answers: A

26. (1 point) Consider an over-parameterized linear regression problem with zero training error solutions and least-squares solution. When optimized using gradient descent initialized at zero, which solution does gradient descent converge to?

- A. The solution with smallest L0 norm
- B. The solution with smallest L2 norm
- C. The solution with largest margin
- D. A random solution depending on initialization
- E. The sparsest solution

Solution:

Gradient descent converges to the minimum L2 norm solution.

Correct: B

27. (1 point) Consider two local minima of a neural network loss: one sharp and one flat. Which statements are correct? (Select all that apply.)

- A. Flat minima are generally more robust to parameter perturbations.
- B. Sharp minima always achieve lower training loss.
- C. Large-batch training is empirically associated with sharper minima.
- D. Flat minima imply lower Hessian eigenvalues.
- E. Sharp minima guarantee poor generalization.

Solution:

Correct: A, C, D

- Flat minima are robust (A).
- Sharp minima do not necessarily have lower training loss (B false).
- Large batches often converge to sharper minima (C).
- Flat minima correspond to small curvature / small eigenvalues (D).
- Sharp minima do not guarantee poor generalization (E false).

28. (2 points) Consider a binary classification model where the output is computed as:

$$\hat{y} = \sigma(z), \quad z = \mathbf{w}^\top \mathbf{x}$$

and the loss is binary cross-entropy:

$$\mathcal{L} = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y}))$$

Instead of treating the sigmoid and the loss as separate nodes in the computation graph, we fuse them into a single operation ("Sigmoid + BCE") during backpropagation.

Which of the following statements are correct? **Select all that apply.**

- A. Fusing the sigmoid and binary cross-entropy simplifies the gradient $\frac{\partial \mathcal{L}}{\partial z}$ to $\hat{y} - y$.
- B. Without fusion, backpropagation requires explicitly computing $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.
- C. Fusion improves numerical stability by avoiding explicit computation of $\log(\sigma(z))$ and $\log(1 - \sigma(z))$.
- D. Fusion changes the mathematical optimum of the model..
- E. Fusion reduces the depth of the computation graph and can improve computational efficiency.

Solution: Correct options: (A), (B), (C), (E)

(A) True. When sigmoid and BCE are fused, the gradient simplifies to:

$$\frac{\partial \mathcal{L}}{\partial z} = \hat{y} - y$$

(B) True. Without fusion, the chain rule requires explicitly computing $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.

(C) True. Fusion allows rewriting the loss in a numerically stable form (e.g., using log-sum-exp tricks), avoiding unstable $\log(\sigma(z))$ terms.

(D) False. Fusion is only a computational optimization and does not change the objective function or its optimum.

(E) True. Fusion reduces graph operations and improves computational efficiency.

29. (2 points) You are training a neural network for image reconstruction. Each pixel value in the target image has been normalized to lie in the range

$$[-1, 1].$$

Which of the following is the most appropriate choice for the **output activation function** of the network? **Select all that apply.**

- A. $\text{ReLU}(z) = \max(0, z)$
- B. $\sigma(z) = \frac{1}{1 + e^{-z}}$
- C. $2\sigma(z) - 1$
- D. $\tanh(z)$
- E. No activation function (linear output)

Solution: Correct choice: (C), (D)

(A) ReLU outputs values in $[0, \infty)$, which does not match the target range.

(B) Sigmoid outputs values in $(0, 1)$, which does not match the target range.

(C) $2\sigma(z) - 1$ maps $(0, 1)$ to $(-1, 1)$, matching the normalized pixel range.

(D) $\tanh(z)$ outputs values in $(-1, 1)$ and matches the normalized pixel range.

(E) A linear output does not constrain the output to $[-1, 1]$, so predictions may fall outside the valid pixel range.

30. (2 points) Consider the Adagrad optimization algorithm. For parameter θ_i at iteration t , the update rule is:

$$g_{t,i} = \nabla_{\theta_i} \mathcal{L}(\theta_t),$$

$$G_{t,i} = \sum_{\tau=1}^t g_{\tau,i}^2,$$

$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,i}} + \epsilon} g_{t,i}.$$

Which of the following statements about Adagrad are correct? **Select all that apply.**

- A. Adagrad uses a different effective learning rate for each parameter.
- B. The effective learning rate for each parameter is non-increasing over time.
- C. Adagrad is particularly well-suited for sparse feature settings.
- D. Adagrad includes an exponential moving average of past gradients.
- E. In long training runs, Adagrad's learning rate may become excessively small.
- F. Do we need to store all the past gradients to calculate the update term in adagrad?

Solution:

Correct choices: (A), (B), (C), (E)

- (A) True. Each parameter has its own accumulated squared gradient $G_{t,i}$.
- (B) True. Since $G_{t,i}$ is a cumulative sum of squares, it is non-decreasing, making $\frac{\eta}{\sqrt{G_{t,i}} + \epsilon}$ non-increasing.
- (C) True. Rarely updated (sparse) parameters accumulate smaller $G_{t,i}$, leading to relatively larger effective learning rates.
- (D) False. Adagrad uses a cumulative sum, not an exponential moving average (that is a property of RMSProp/Adam).
- (E) True. Because $G_{t,i}$ keeps growing, the effective learning rate may decay too aggressively in long training.
- (F) False. Adagrad only stores the running sum $G_{t,i}$, not all past gradients.

31. (1 point) A dataset contains N training examples. A neural network is trained using mini-batch gradient descent with batch size B for n epochs.

Assume:

- Each mini-batch requires exactly b seconds for one forward + backward pass.
- N is divisible by B .
- The time for validation, logging, and data loading is negligible.

If $N = 50,000$, $B = 100$, $n = 20$, and $b = 0.25$ seconds, compute the total training time.

Solution:

(a) Derivation

Number of mini-batches per epoch:

$$\frac{N}{B}$$

Total number of mini-batch updates over n epochs:

$$n \cdot \frac{N}{B}$$

Since each update takes b seconds:

$$T = n \cdot \frac{N}{B} \cdot b$$

(b) Numerical computation

$$\frac{N}{B} = \frac{50,000}{100} = 500$$

Total number of updates:

$$20 \times 500 = 10,000$$

Total time:

$$T = 10,000 \times 0.25 = 2,500 \text{ seconds}$$

$$T = 2500 \text{ seconds}$$

32. (1 point) A fully connected neural network has:

Input layer: 20 units

Hidden layer: 10 units

Output layer: 5 units

How many total trainable parameters (including biases) does the network have?

A. 255

B. 265

- C. 275
- D. 285

Solution: Correct Answer: B

Input $\rightarrow$ Hidden:

Weights: $20 \times 10 = 200$

Biases: 10

Hidden $\rightarrow$ Output:

Weights: $10 \times 5 = 50$

Biases: 5

Total parameters:

$$200 + 10 + 50 + 5 = 265$$

33. (1 point) In a computational graph, suppose a node feeds into two different branches.

During backpropagation, how are gradients combined at that node?

- A. Only one gradient is used
- B. Gradients are multiplied
- C. Gradients are summed
- D. Gradients cancel out

Solution: Correct Answer: C

Suppose a node x feeds into two branches that compute intermediate variables $u(x)$ and $v(x)$, and the final loss is

$$L = L(u(x), v(x)).$$

By the multivariable chain rule,

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial u} \frac{\partial u}{\partial x} + \frac{\partial L}{\partial v} \frac{\partial v}{\partial x}.$$

Thus, the total gradient with respect to x is the *sum* of gradient contributions from all outgoing branches.

Therefore, gradients are summed.

34. (1 point) Which of the following statements are correct? (Select all that apply.)

- A. Adam uses adaptive learning rates
- B. RMSProp uses exponential moving average of squared gradients
- C. SGD with momentum uses second-order derivatives
- D. Adagrad maintains an exponential moving average of squared gradients
- E. Nesterov Accelerated Gradient evaluates gradient at the current parameter location

Solution: Correct Answers: A, B

A. Adam uses adaptive learning rates:

Adam maintains first and second moment estimates:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t,$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

Each parameter is scaled by its own $\sqrt{\hat{v}_t}$, making the learning rate adaptive. Hence this statement is correct.

B. RMSProp uses exponential moving average of squared gradients:

RMSProp maintains

$$v_t = \beta v_{t-1} + (1 - \beta) g_t^2,$$

which is an exponential moving average of squared gradients. Hence this statement is correct.

C. SGD with momentum uses second-order derivatives:

Momentum uses only first-order gradients. No Hessian or second-order derivatives appear. Hence this statement is false.

D. Adagrad maintains an exponential moving average of squared gradients:

Adagrad accumulates gradients as

$$G_t = \sum_{i=1}^t g_i^2,$$

which is a cumulative sum, not an exponential moving average. Hence this statement is false.

E. Nesterov Accelerated Gradient evaluates gradient at the current parameter location:

NAG evaluates the gradient at a look-ahead position:

$$\theta_t - \eta \mu v_{t-1}.$$

Hence this statement is false.

35. (1 point) Consider gradient descent applied to the loss:

$$L(w) = \frac{1}{2}\alpha w^2$$

with update rule:

$$w_{t+1} = w_t - \eta\alpha w_t.$$

For which condition does the **loss** converge to zero?

- A. $\eta\alpha > 2$
- B. $0 < \eta\alpha < 2$
- C. $\eta\alpha < 0$
- D. Loss always diverges

Solution: Correct Answer: B

Update:

$$w_{t+1} = (1 - \eta\alpha)w_t$$

Loss evolves as:

$$L_t \propto (1 - \eta\alpha)^{2t}$$

Convergence requires:

$$|1 - \eta\alpha| < 1$$

$$0 < \eta\alpha < 2$$